Heart Disease Prediction – MLOps Pipeline

Overview

This project implements an end-to-end MLOps pipeline for heart disease prediction using the KDD Cup 1999–style dataset (binary classification: normal vs attack). The pipeline covers data preprocessing, EDA, model training, experiment tracking with MLflow, containerization with Docker, deployment on Kubernetes (Minikube), logging, and simple monitoring via Prometheus-style metrics.

1. Setup and Installation

1.1 Prerequisites

- Python 3.x
- pip
- Docker
- Minikube
- kubectl
- Git

1.2 Setup & Installation — Action/Command Table

Setup Step	Command(s) to Run
Clone the repository	git clone https://github.com/subhasajayan/mlops-heart-disease cd mlops-heart-disease
Create and activate virtual environment	python -m venv venv venv\Scripts\activate
Install dependencies	pip install -r requirements.txt
Run experiments locally	python src/train.py
Build Docker image	docker build -t heart-api .
Start Minikube	minikube start
Load Docker image into Minikube	minikube image load heart-api
Apply Kubernetes deployment	kubectl apply -f k8s/deployment.yaml
Apply Kubernetes service	kubectl apply -f k8s/service.yaml
Get service URL	minikube service heart-api-service --url

2. EDA & Modelling Choices

2.1 Dataset Preparation

The project uses the KDD-style heart disease dataset, originally provided in .dat format. To ensure reproducibility and ease of use, the repository includes:

- A **Python script to download the** .dat **file** directly from the source.
  - data\script\download_data.py
- A **Python conversion script** that transforms the .dat file into a structured .csv file.
  - src\preprocess.py
- Automated preprocessing steps inside the training pipeline to load the .csv file for EDA and modelling.
- On top in data folder contains both .dat and csv file ready to use
  - data\processed\heart_clean.csv
  - data\raw\heart.dat

This ensures that anyone running the project can reproduce the dataset preparation without manual intervention.

2.2 Exploratory Data Analysis (EDA)

Key findings:

- Dataset contains 270 rows and 14 columns.
- No missing values.
- Balanced target classes (150 vs 120).
- Histograms show reasonable distributions.
- Correlation heatmap highlights strong predictors (thalach, oldpeak, cp, slope).

2.3 Feature Engineering

Using ColumnTransformer:

- Numeric features scaled using StandardScaler.
- Categorical features encoded using OneHotEncoder.
- Unified preprocessing pipeline ensures consistent transformations.

2.4 Train/Test Split

- 80/20 split.
- Stratified to preserve class balance.
- Shapes: Train (216), Test (54).

2.5 Model Development

Two models were trained:

- Logistic Regression (max_iter=1000)
- Random Forest (n_estimators=200, random_state=42)

2.6 Model Evaluation

Logistic Regression

- Accuracy: 0.8703
- ROC-AUC: 0.9139

Random Forest

- Accuracy: 0.8333
- ROC-AUC: 0.8875

Conclusion: Logistic Regression performed better.

2.7 Cross-Validation

- Logistic Regression Mean ROC-AUC: 0.9128
- Random Forest Mean ROC-AUC: 0.9113

2.8 Final Model Selection

Final model: **Logistic Regression** Saved as best_model.pkl and preprocessor.pkl.

---

3. Experiment Tracking Summary (MLflow)

3.1 MLflow Integration

MLflow was used to track:

- Hyperparameters
- Metrics (accuracy, precision, recall, F1, ROC-AUC)
- Artifacts (ROC curve, confusion matrix, saved models)
- Run history for comparison

Tracking URI: sqlite:///mlflow.db Experiment name: heart_disease_classification

3.2 Logged Artifacts

- ROC Curve
- Confusion Matrix
- Saved model (MLflow format)
- Preprocessing pipeline

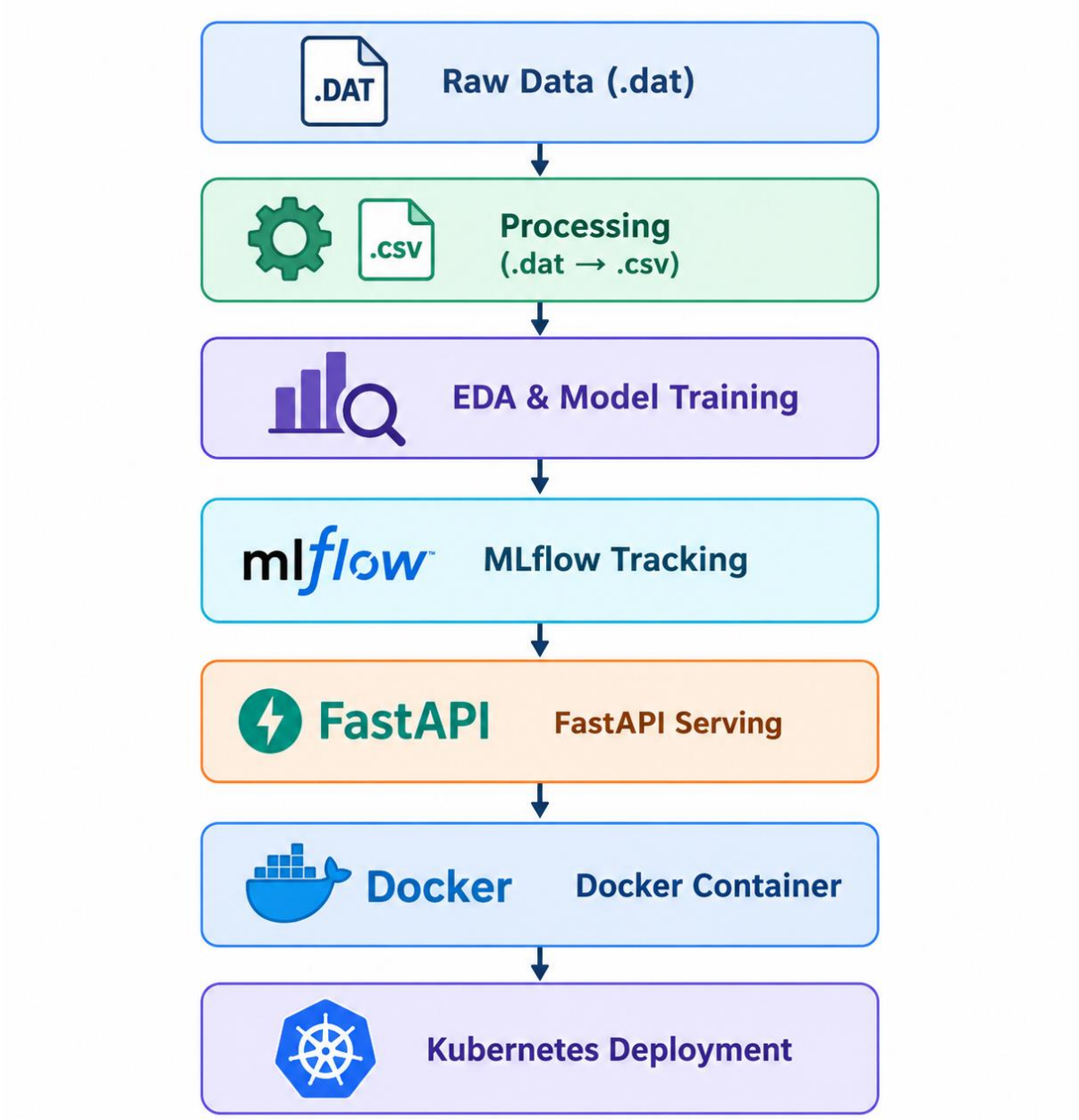
3.3 MLflow UI

Launch using:

Step	Command
Start MLflow UI	mlflow ui --port 5001

4. Architecture Diagram (Text-Based)

4.1 System Architecture Overview



4.2 Explanation

- Data flows from raw .dat → cleaned .csv.
- EDA + training produce the final model.
- MLflow logs experiments.
- FastAPI serves predictions.
- Docker containerizes the API.
- Kubernetes deploys the service.
- Minikube exposes the endpoint.

5. CI/CD & Deployment Workflow

5.1 CI/CD Pipeline

A GitHub Actions workflow includes:

- Linting
- Unit tests
- Model training
- Artifact upload
- Deployment triggers

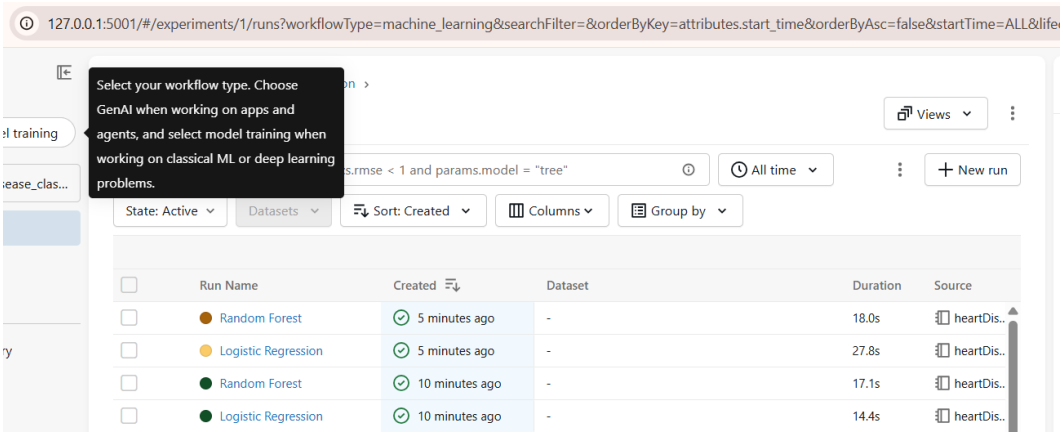
5.2 Deployment Workflow

Step	Command
Build Docker image	docker build -t heart-api .
Load image into Minikube	minikube image load heart-api
Apply deployment	kubectl apply -f k8s/deployment.yaml
Apply service	kubectl apply -f k8s/service.yaml
Get service URL	minikube service heart-api-service --url

---

6. Screenshot Titles

6.1 MLflow UI – Experiment List



Metrics (5)

Search metrics

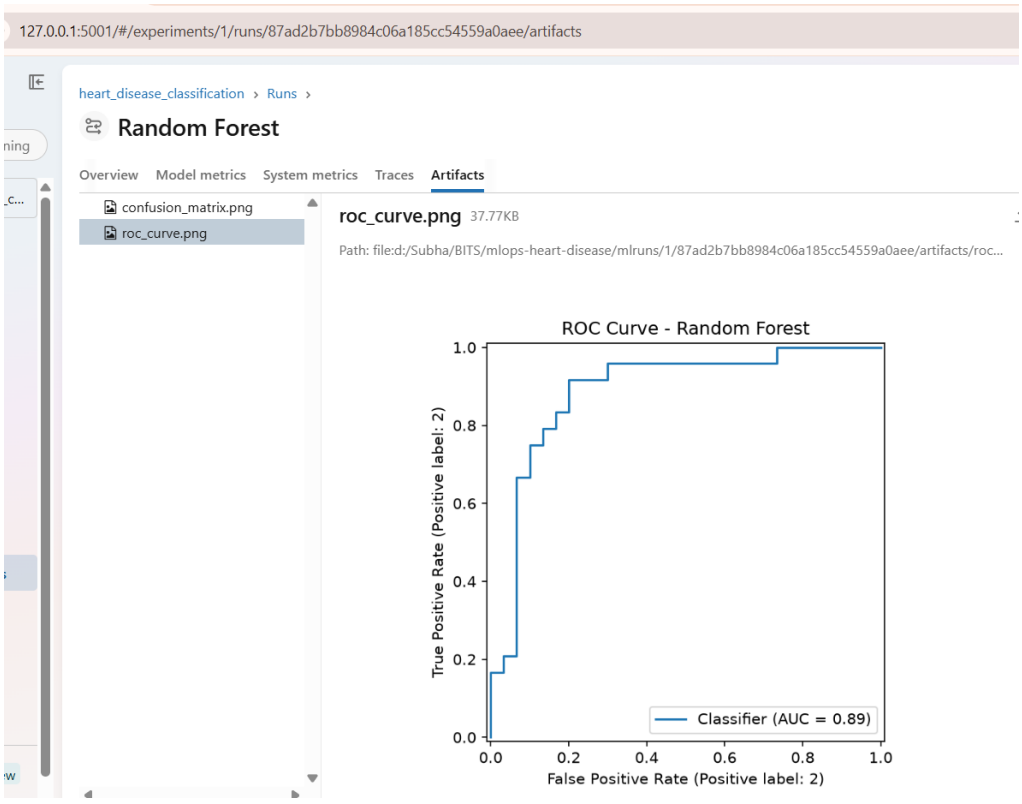
Metric	Value	Models
accuracy	0.8703703703703703	Logistic Regression
precision	0.9259259259259259	Logistic Regression
recall	0.8333333333333334	Logistic Regression
f1_score	0.8771929824561403	Logistic Regression
roc_auc	0.9138888888888889	Logistic Regression

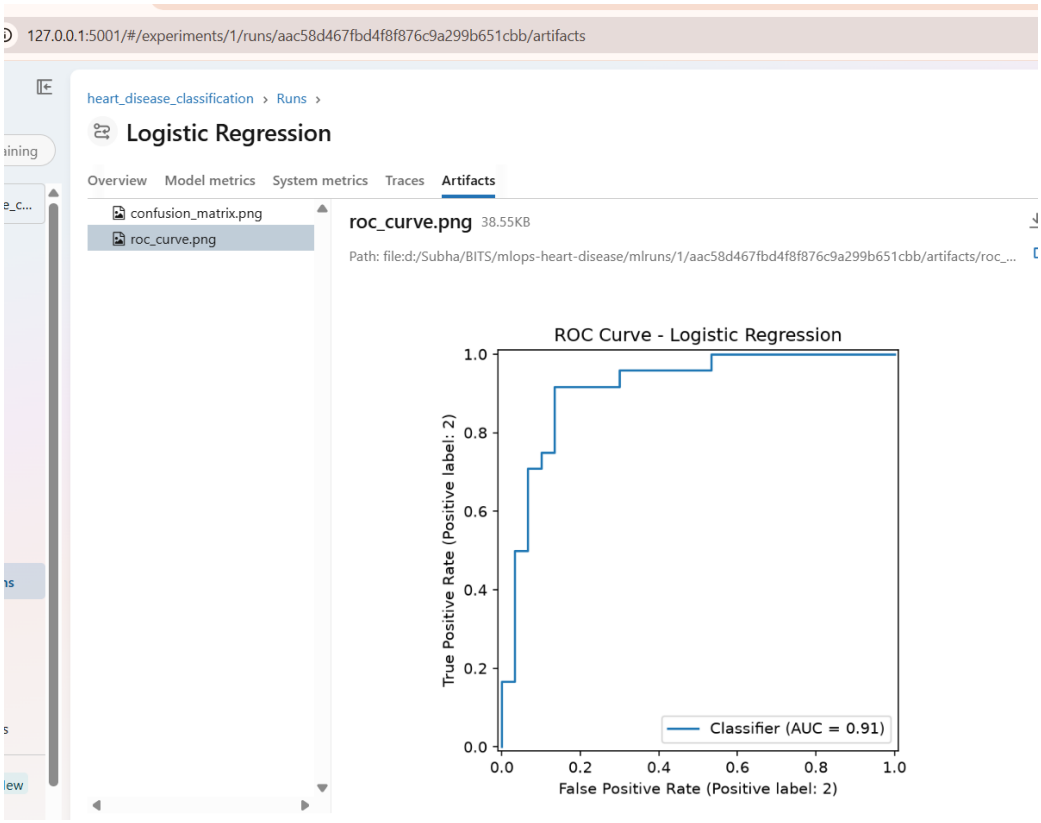
Metrics (5)

Search metrics

Metric	Value	Models
accuracy	0.8333333333333334	Random Forest
precision	0.8620689655172413	Random Forest
recall	0.8333333333333334	Random Forest
f1_score	0.847457627118644	Random Forest
roc_auc	0.8875000000000001	Random Forest

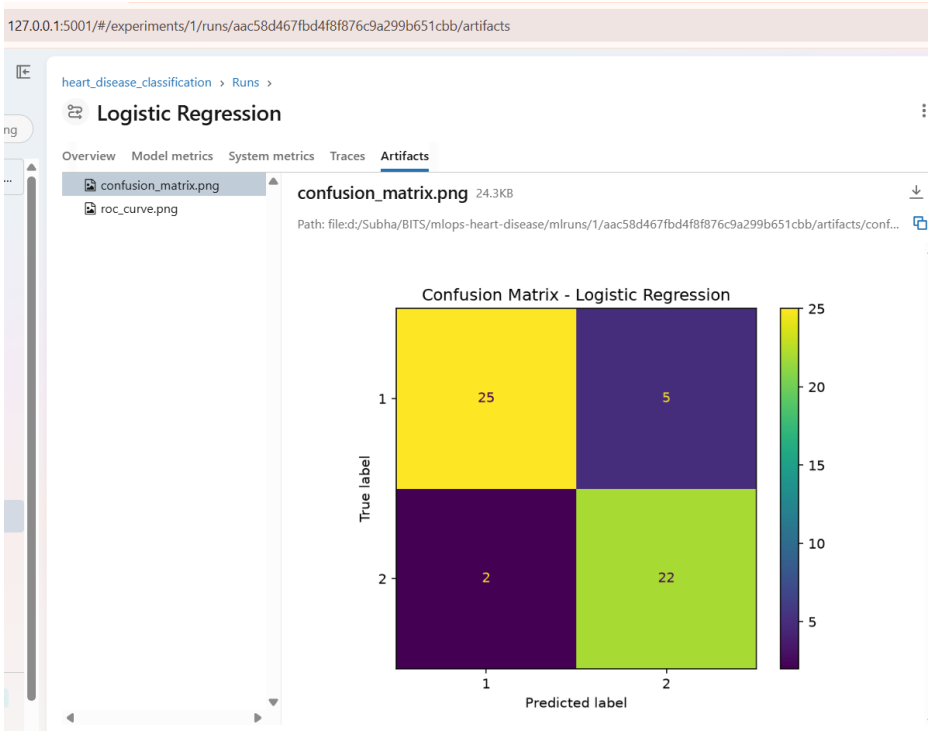
6.2 MLflow – ROC Curve Artifact





6.3 MLflow – Confusion Matrix Artifact

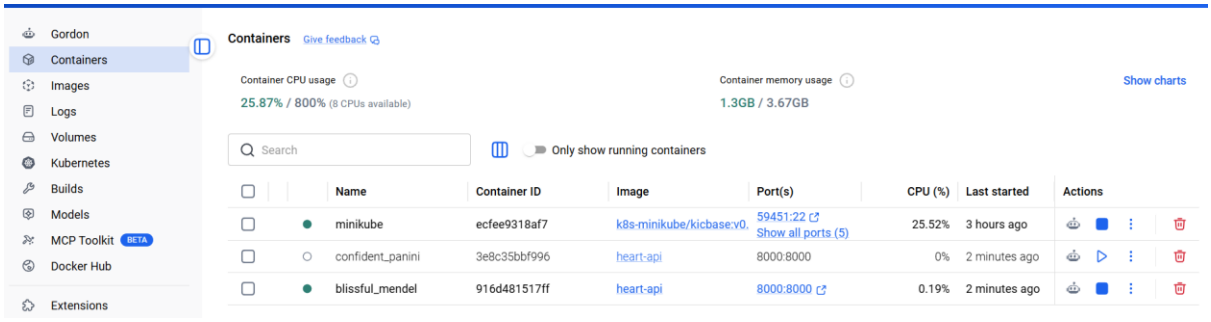




6.4 Docker Build Success

```
D:\Subha\BITS\mlops-heart-disease>docker run -p 8000:8000 heart-api
```

```
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```



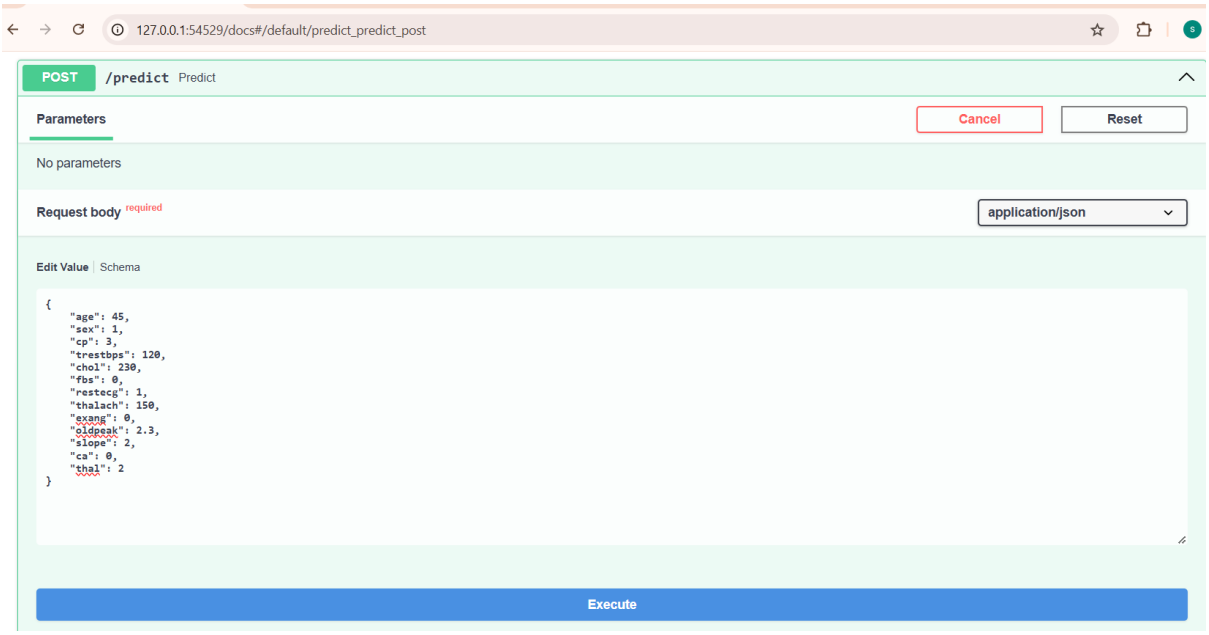
6.5 Kubernetes Deployment (kubectl get pods)

```
PS D:\Subha\BITS\mlops-heart-disease> kubectl get pods
No resources found in default namespace.
PS D:\Subha\BITS\mlops-heart-disease> kubectl apply -f k8s/deployment.yaml
deployment.apps/heart-api-deployment created
PS D:\Subha\BITS\mlops-heart-disease> kubectl get deployments
NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
heart-api-deployment   1/1     1            1           76s
PS D:\Subha\BITS\mlops-heart-disease> minikube image load heart-api
PS D:\Subha\BITS\mlops-heart-disease> kubectl delete deployment heart-api-deployment
deployment.apps "heart-api-deployment" deleted from default namespace
PS D:\Subha\BITS\mlops-heart-disease> kubectl apply -f k8s/deployment.yaml
deployment.apps/heart-api-deployment created
PS D:\Subha\BITS\mlops-heart-disease> kubectl get pods
NAME                                 READY   STATUS    RESTARTS   AGE
heart-api-deployment-5f7589986b-gwgw2   1/1     Running   0          11s
PS D:\Subha\BITS\mlops-heart-disease> minikube service heart-api-service --url
http://127.0.0.1:63333
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

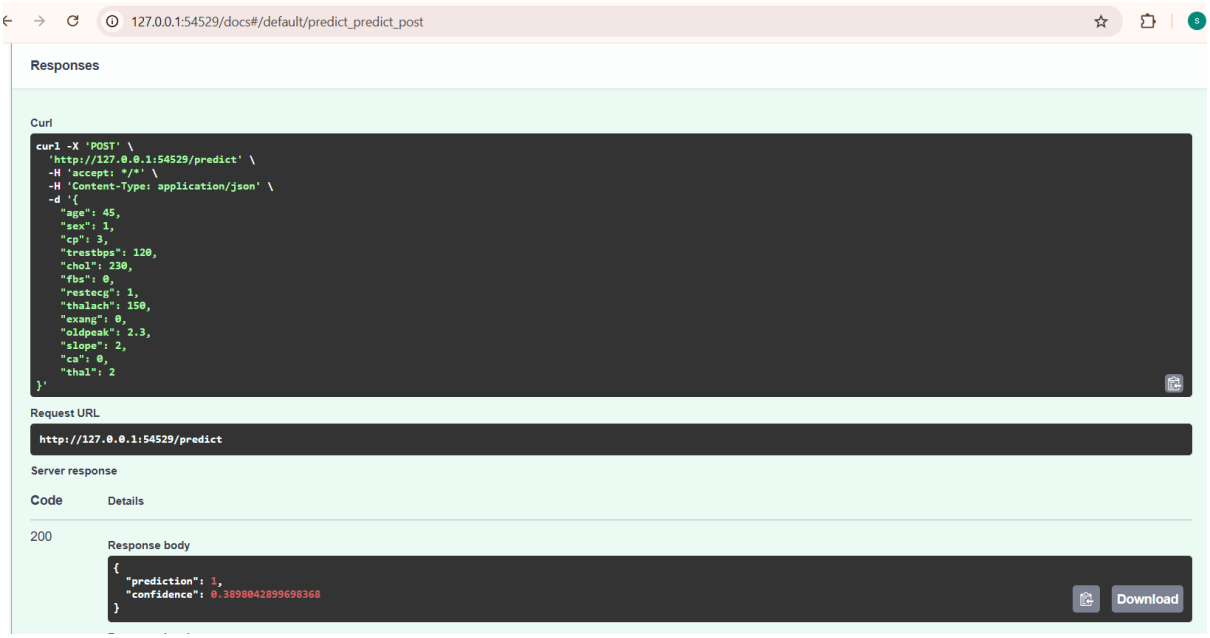
6.6 Kubernetes Service (kubectl get svc)

```
C:\Users\sajay>kubectl get svc
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
heart-api-service   LoadBalancer   10.106.220.124   <pending>     8000:31229/TCP   3h15m
kubernetes          ClusterIP      10.96.0.1        <none>        443/TCP          3h25m
```

6.7 FastAPI Swagger UI (/docs)



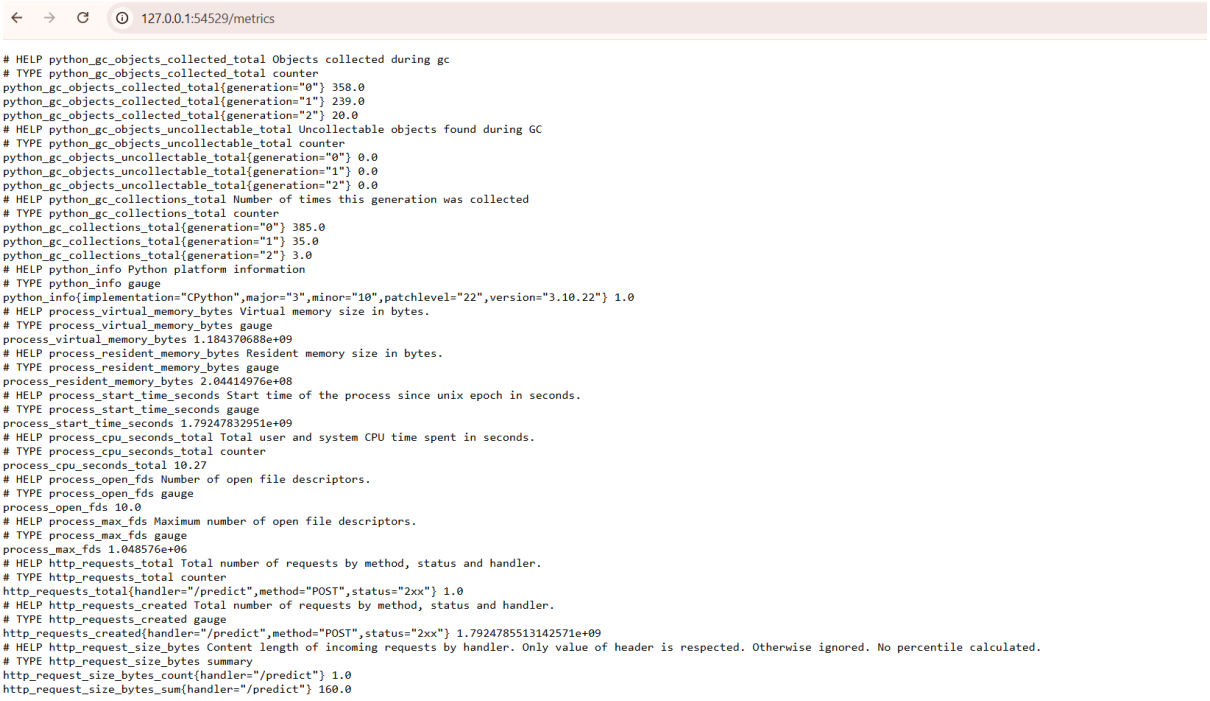
6.8 Prediction Response (/predict)



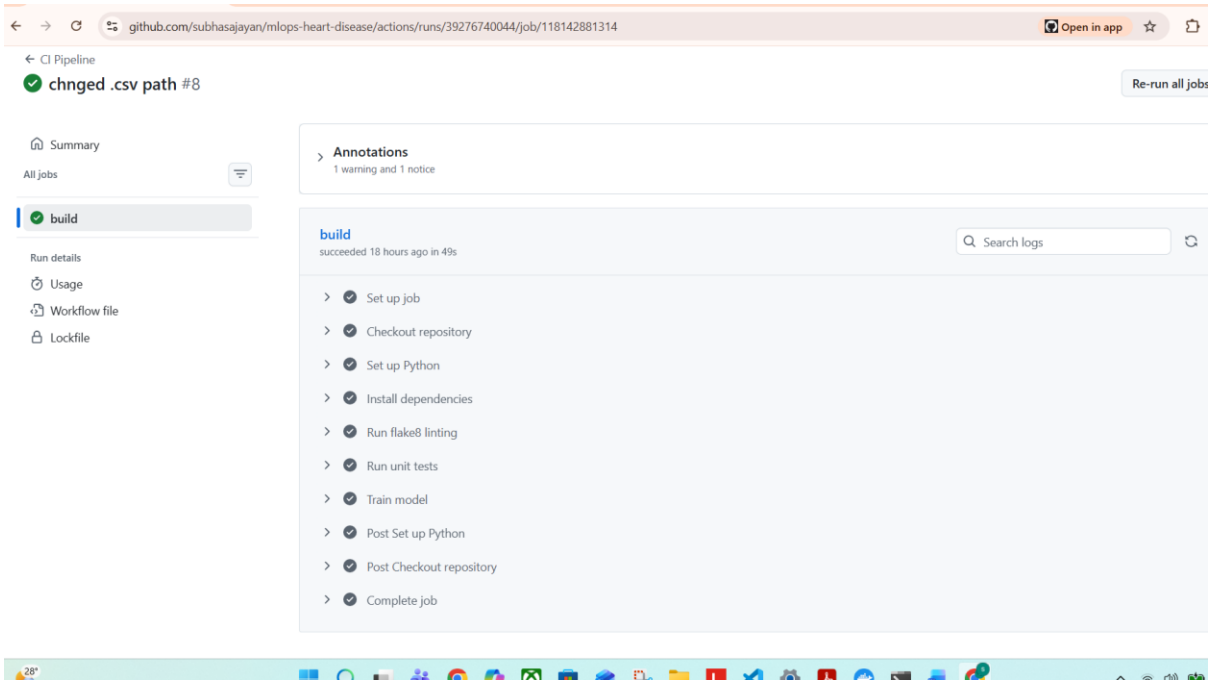
6.8 Logs (/predict)

```
C:\Users\sajay>kubectl logs heart-api-deployment-8dcf8887d-2k7xj
```

```
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
2026-10-20 06:42:28,555 - INFO - Incoming request: POST /predict
2026-10-20 06:42:31,311 - INFO - Completed request: POST /predict Status: 200 Time: 2760.35 ms
INFO:     10.244.0.1:10470 - "POST /predict HTTP/1.1" 200 OK
```



6.8 Git workflow action



7. Repository Link

GitHub Repository: https://github.com/subhasajayan/mlops-heart-disease

---

8. Conclusion

This project demonstrates a complete MLOps workflow for heart disease prediction, including data acquisition, EDA, feature engineering, model development, experiment tracking, containerization, and Kubernetes deployment. The pipeline is fully reproducible, modular, and ready for extension into production environments.